

Author: Pankaj Yadav

Date: 04/12/2026

Description:

apply clustering methods on anonymized ALS patients

Week 4 Assignment

You will be using the dataset als_data.csv [https://content.bellevue.edu/cst/dsc/630/als_data.csv] to apply clustering methods for this assignment. This data gives anonymized data on ALS patients. With this data, complete the following steps:

Remove any data that is not relevant to the patient's ALS condition.

Apply a standard scalar to the data.

Create a plot of the cluster silhouette score versus the number of clusters in a K-means cluster.

Use the plot created in (3) to choose an optimal number of clusters for K-means. Justify your choice.

Fit a K-means model to the data with the optimal number of clusters chosen in part (4).

Fit a PCA transformation with two features to the scaled data.

Make a scatterplot the PCA transformed data coloring each point by its cluster value.

Summarize your results and make a conclusion.

IMPORTANT: Please make sure to follow the following submission instructions carefully or it will result in having to resubmit or losing points if repeated incorrect submissions are made.

```
In [27]: # Import Libraries
import pandas as pd
from sklearn.preprocessing import StandardScaler
import numpy as np
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
import seaborn as sns
```

Clustering Analysis on ALS Patient Data

In this section, I will apply clustering methods to the ALS patient dataset as outlined in the assignment. The goal is to identify meaningful patient groupings that could help in understanding patterns in ALS conditions.

Import the data

The first step to generate insights is to understand your data. And to understand the data you not just have to understand the variables but also the story behind that data. I still believe pandas a good in reading data, hence using read_csv function from pandas to read csv file in to a data frame and using head function to read the first 5 rows from it.

```
In [14]: # Load the dataset
als_data = pd.read_csv('data//als_data.csv')

# display first few rows to check if data is loaded correctly
als_data.head(5)
```

```
Out[14]:
```

	ID	Age_mean	Albumin_max	Albumin_median	Albumin_min	Albumin_range	ALSFRS_slope	ALSFRS_Total_max	ALSFRS_Total_me
0	1	65	57.0	40.5	38.0	0.066202	-0.965608	30	
1	2	48	45.0	41.0	39.0	0.010453	-0.921717	37	
2	3	38	50.0	47.0	45.0	0.008929	-0.914787	24	
3	4	63	47.0	44.0	41.0	0.012111	-0.598361	30	
4	5	63	47.0	45.5	42.0	0.008292	-0.444039	32	

5 rows x 101 columns

The ALS dataset used in this analysis contains detailed clinical and laboratory information for each patient, with each row representing a unique individual. The columns include a variety of features such as demographic data (like average age), summary statistics for lab measurements (for example, maximum, minimum, median, and range values for markers like Albumin, Sodium, and Urine pH), and clinical assessments related to ALS progression, such as the ALS Functional Rating Scale scores and their rate of change. There are also columns capturing specific functional abilities (e.g., trunk strength) summarized in similar ways. Some columns, such as patient IDs, serve only as identifiers and do not provide clinical insight. For clustering and further analysis, the focus will be on the features that directly reflect the patient's ALS condition and disease progression, while non-informative identifiers will be excluded to ensure meaningful groupings. This rich dataset structure allows for a comprehensive exploration of patterns and subgroups within the ALS patient population.

```
In [15]: # Inspect the columns to identify irrelevant features
als_data.info()
als_data.head()

# Drop columns not relevant to ALS condition (e.g., IDs, names, unrelated metadata)
```

```
# Adjust the list below based on actual column names in als_data
irrelevant_cols = ['PatientID', 'Name', 'ID', 'SubjectID'] # Example placeholders
als_data_clean = als_data.drop(columns=[col for col in irrelevant_cols if col in als_data.columns])

# Display cleaned data
als_data_clean.head()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2223 entries, 0 to 2222
Columns: 101 entries, ID to Urine.Ph_min
dtypes: float64(75), int64(26)
memory usage: 1.7 MB
```

```
Out[15]:
```

	Age_mean	Albumin_max	Albumin_median	Albumin_min	Albumin_range	ALSFRS_slope	ALSFRS_Total_max	ALSFRS_Total_median
0	65	57.0	40.5	38.0	0.066202	-0.965608	30	28.0
1	48	45.0	41.0	39.0	0.010453	-0.921717	37	33.0
2	38	50.0	47.0	45.0	0.008929	-0.914787	24	14.0
3	63	47.0	44.0	41.0	0.012111	-0.598361	30	29.0
4	63	47.0	45.5	42.0	0.008292	-0.444039	32	27.5

5 rows x 99 columns

Lets do some basic EDA on data before moving forward with data standardization. first check if there are missing values and provide a summary statistics.

```
In [16]: als_data_clean.describe()
```

Out [16]:

	Age_mean	Albumin_max	Albumin_median	Albumin_min	Albumin_range	ALSFRS_slope	ALSFRS_Total_max	ALSFRS_Total_r
count	2223.000000	2223.000000	2223.000000	2223.000000	2223.000000	2223.000000	2223.000000	2223.0
mean	54.550157	47.011134	43.952542	40.766347	0.013779	-0.728274	31.692308	27.7
std	11.396546	3.233980	2.654804	3.193087	0.009567	0.622329	5.314228	6.6
min	18.000000	37.000000	34.500000	24.000000	0.000000	-4.345238	11.000000	2.5
25%	47.000000	45.000000	42.000000	39.000000	0.009042	-1.086310	29.000000	23.0
50%	55.000000	47.000000	44.000000	41.000000	0.012111	-0.620748	33.000000	28.0
75%	63.000000	49.000000	46.000000	43.000000	0.015873	-0.283832	36.000000	32.0
max	81.000000	70.300000	51.100000	49.000000	0.243902	1.207011	40.000000	40.0

8 rows × 99 columns

There are no missing values as all features have 2223 rows. There are no outliers when you see max vs mean and 75th percentile values. This means the dataset is now well-prepared for scaling and clustering. All remaining columns represent meaningful patient attributes, such as age, various lab values (e.g., Albumin, Sodium), ALSFRS scores, and functional measures (e.g., trunk strength). These features will allow the clustering algorithm to group patients based on their clinical profiles and disease progression, as intended by the assignment.

As discussed in Chapter 5, it is important to standardize the features so that each contributes equally to the distance calculations. I will use StandardScaler from scikit-learn to scale the data. As Clustering algorithms like K-means are sensitive to the scale of the data, its a good idea to standardize the features.

```
In [ ]: # Apply standard scalar
scaler = StandardScaler()
als_scaled = scaler.fit_transform(als_data_clean)
```

After applying standard scaling with StandardScaler, the output is a NumPy array, which does not retain the original column names or index from your DataFrame. Lets convert it back to a Data frame to keep the original column names for easier interpretation.

```
In [18]: # Convert back to DataFrame for easier handling
als_scaled_df = pd.DataFrame(als_scaled, columns=als_data_clean.columns)
als_scaled_df.head()
```

Out [18]:

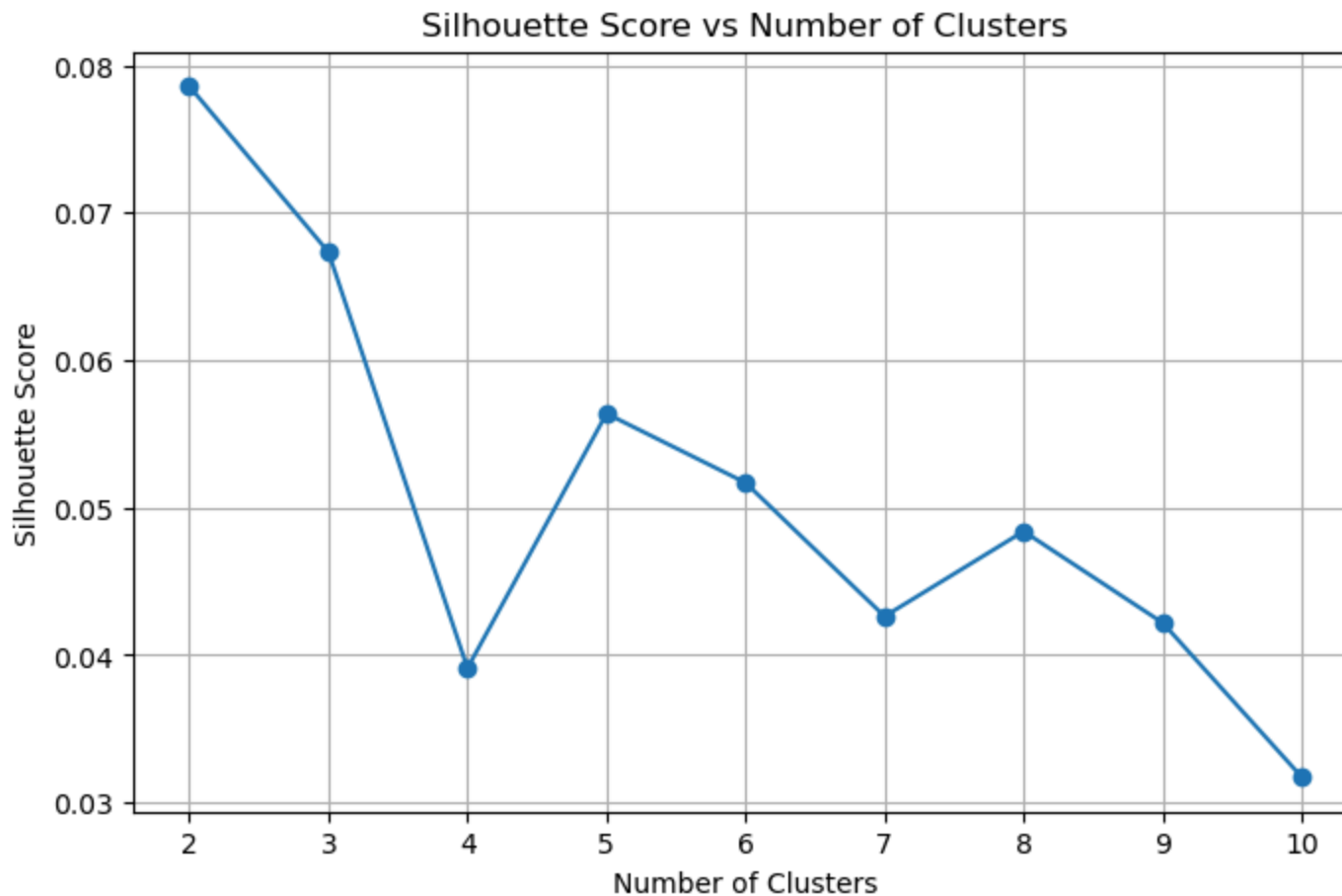
	Age_mean	Albumin_max	Albumin_median	Albumin_min	Albumin_range	ALSFRS_slope	ALSFRS_Total_max	ALSFRS_Total_median
0	0.917137	3.089417	-1.300781	-0.866550	5.480929	-0.381450	-0.318520	0.134960
1	-0.574879	-0.622016	-1.112401	-0.553303	-0.347725	-0.310907	0.998995	0.888863
2	-1.452535	0.924415	1.148162	1.326179	-0.507103	-0.299769	-1.447819	-1.975969
3	0.741606	-0.003443	0.017880	0.073191	-0.174361	0.208801	-0.318520	0.285741
4	0.741606	-0.003443	0.583021	0.386438	-0.573670	0.456831	0.057913	0.059570

5 rows × 9 columns

Now as we have the data frame back, lets create a plot of the cluster silhouette score versus the number of clusters in a K-means cluster. Silhouette score measures how well each data point fits within its cluster. Its used to find the best number of clusters. As recommended in the readings, I will plot the silhouette score for a range of cluster counts to visually select the optimal value.

```
In [22]: # Test different numbers of clusters and record silhouette scores
silhouette_scores = []
cluster_range = range(2, 11)
for k in cluster_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(als_scaled_df)
    score = silhouette_score(als_scaled_df, labels)
    silhouette_scores.append(score)
optimal_k = cluster_range[np.argmax(silhouette_scores)]

# Plot silhouette scores
plt.figure(figsize=(8, 5))
plt.plot(cluster_range, silhouette_scores, marker='o')
plt.title('Silhouette Score vs Number of Clusters')
plt.xlabel('Number of Clusters')
plt.ylabel('Silhouette Score')
plt.grid(True)
plt.show()
```



The plot suggests that the ALS patient data naturally separates into 2 main clusters, with additional clusters not providing better-defined groupings. This helps you justify your choice of the number of clusters for the next steps in your analysis. The score decreases as the number of clusters increases, with a small local peak at $k=5$. For $k > 2$, the scores are lower and fluctuate, suggesting that additional clusters do not improve the separation and may even make the clustering less meaningful. So 2 is the optimal number with best score.

With the optimal number of clusters determined as 2, I will fit the K-means model to the scaled data and assign cluster labels to each patient.

```
In [23]: # Fit K-means with the optimal number of clusters
kmeans_final = KMeans(n_clusters=optimal_k, random_state=42)
cluster_labels = kmeans_final.fit_predict(als_scaled_df)

# Add cluster labels to the DataFrame
```

```
als_scaled_df['Cluster'] = cluster_labels
als_scaled_df.head()
```

Out [23]:

	Age_mean	Albumin_max	Albumin_median	Albumin_min	Albumin_range	ALSFRS_slope	ALSFRS_Total_max	ALSFRS_Total_median
0	0.917137	3.089417	-1.300781	-0.866550	5.480929	-0.381450	-0.318520	0.134960
1	-0.574879	-0.622016	-1.112401	-0.553303	-0.347725	-0.310907	0.998995	0.888863
2	-1.452535	0.924415	1.148162	1.326179	-0.507103	-0.299769	-1.447819	-1.975969
3	0.741606	-0.003443	0.017880	0.073191	-0.174361	0.208801	-0.318520	0.285741
4	0.741606	-0.003443	0.583021	0.386438	-0.573670	0.456831	0.057913	0.059570

5 rows × 100 columns

To visualize the clusters, I will use Principal Component Analysis (PCA) to reduce the data to two dimensions. This technique, as described in the readings, helps in understanding the structure of high dimensional data.

The PCA function reduce the high dimensional scaled ALS data to two principal components PC1 & PC2. Two components are chosen so the data can be visualized in a 2D scatter plot. fit_transform fits PCA to the scaled data and transform it. It projects each patient in to a new 2D space that captures the most variance in the data for that component.

In the second part we just convert the array back to the data frame

```
In [26]: # Fit PCA to reduce to 2 components
pca = PCA(n_components=2, random_state=42)
als_pca = pca.fit_transform(als_scaled_df.drop('Cluster', axis=1))

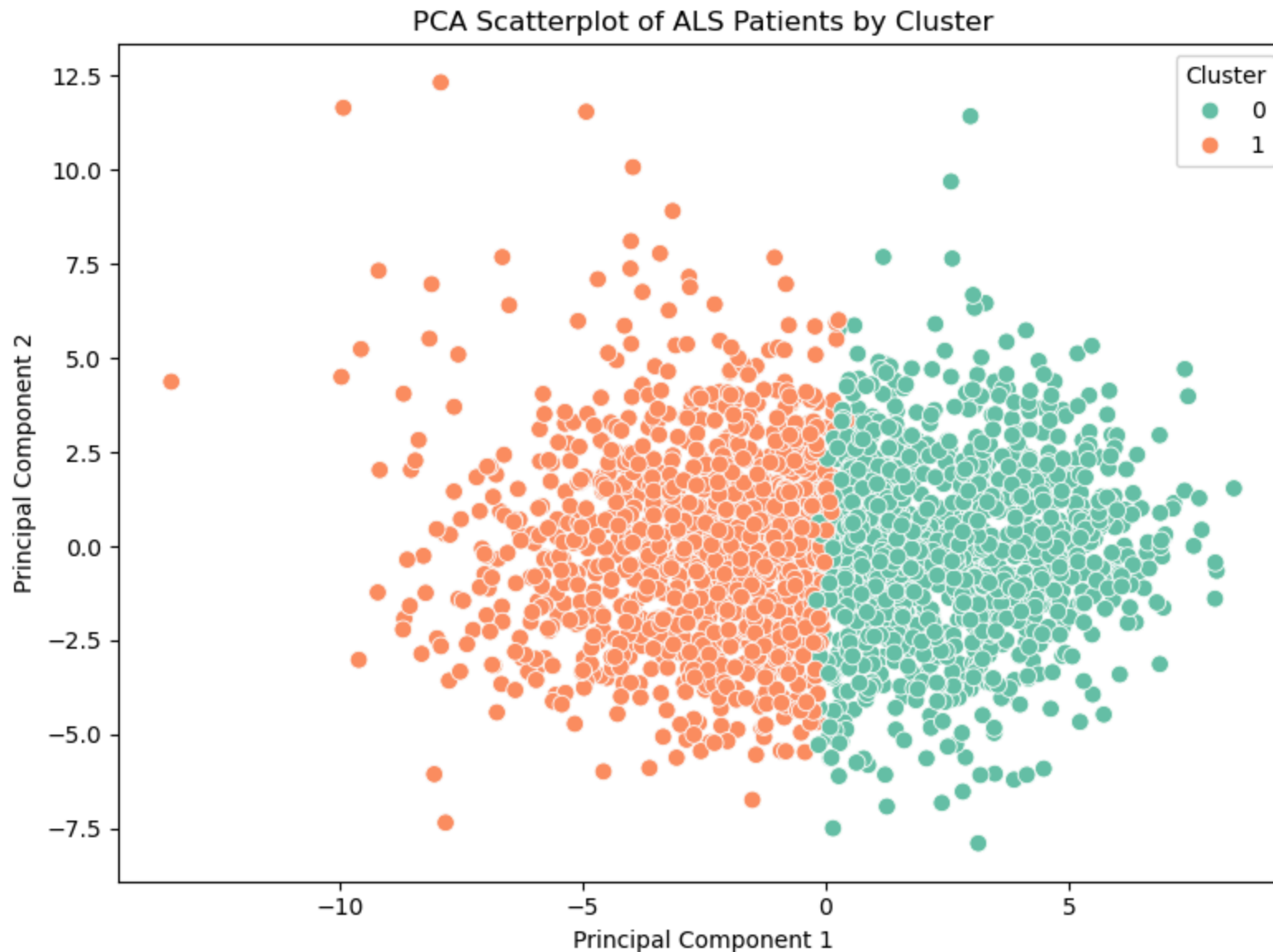
# Create a DataFrame for plotting
als_pca_df = pd.DataFrame(als_pca, columns=['PC1', 'PC2'])
als_pca_df['Cluster'] = cluster_labels
als_pca_df.head()
```

Out [26]:

	PC1	PC2	Cluster
0	1.426731	-2.319374	0
1	1.440235	-4.871672	0
2	-1.617855	-0.429015	1
3	1.919998	2.095630	0
4	-0.297704	0.167252	1

Now I will create a scatterplot of the PCA-transformed data, coloring each point by its cluster assignment. This visualization helps to see how well the clusters are separated in two dimensions.

```
In [28]: plt.figure(figsize=(8, 6))
sns.scatterplot(data=als_pca_df, x='PC1', y='PC2', hue='Cluster', palette='Set2', s=60)
plt.title('PCA Scatterplot of ALS Patients by Cluster')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend(title='Cluster')
plt.tight_layout()
plt.show()
```



The PCA scatterplot above visualizes the ALS patient data after reducing its high dimensionality to two principal components (PC1 and PC2). Each point represents a patient, and the color indicates the cluster assignment from K-means clustering with two clusters, as determined optimal by the silhouette score. The plot shows a clear separation between the two clusters along the first principal component, suggesting that the clustering algorithm was able to distinguish two main groups within the patient population. While there is a little overlap, the clusters are generally well-separated, indicating that the features used in the analysis capture meaningful differences among patients.

Summary:

Through a systematic clustering analysis of the ALS patient dataset, I first performed exploratory data analysis to ensure data quality and relevance, removing identifiers and confirming the absence of missing values or outliers. After standardizing the features, I used the silhouette score to determine the optimal number of clusters for K-means, which was found to be two. The clustering was then visualized using PCA, which reduced the data to two principal components for easier interpretation. The resulting scatterplot showed that the two clusters are reasonably well-separated, supporting the validity of the clustering solution.

Conclusion:

The analysis reveals that the ALS patient data naturally divides into two main clusters, as indicated by both the silhouette score and the PCA visualization. These clusters likely represent distinct subgroups within the ALS population, potentially reflecting differences in disease progression, clinical characteristics, or response to treatment. The clear separation in the PCA plot suggests that the features selected for clustering are meaningful and capture important aspects of patient heterogeneity. This clustering approach, grounded in the principles from the course readings, provides a foundation for further investigation into the characteristics and clinical significance of each subgroup, which could ultimately inform more personalized approaches to ALS care and research.